

Origami bundler

Audit Report

prepared by Pavel Anokhin ([@panprog](#))

November 11, 2025

Version: 2

Contents

1. About Guardefy and @panprog	2
2. Disclaimer	2
3. Scope of the audit	2
4. Audit Timeline	2
5. Findings	4
5.1. High severity findings	4
5.2. Medium severity findings	4
5.3. Low severity findings	4
5.3.1. [L-1] Malicious bundle can block all unapprove attempts by reverting <code>OrigamiBundlerPluginMultiAccess.setBundlerApproved</code> in the <code>supportsInterface</code> function.	4
5.3.2. [L-2] Setting vault untrusted in TBS plugin <code>setTrustedVault</code> can fail and revert for malicious tokens or too many tokens added.	5
5.4. Informational severity findings	6
5.4.1. [I-1] When TBS plugins set vault to untrusted, the <code>_approvedTokens</code> set is not cleared.	6
5.4.2. [I-2] Kyber Swap plugin <code>swapBalance</code> function might not always revert when scaling input amount fails.	7
5.4.3. [I-3] Suggestion: <code>erc20TransferBalanceFromInitiator</code> and <code>permit2TransferBalanceFromInitiator</code> functions of Entry Point plugin will revert if there is not enough approval to transfer all balance.	7
5.4.4. [I-4] Typo in comment in <code>PendleRouterScalingLib</code>	8

1. About Guardefy and @panprog

Pavel Anokhin or **@panprog**, doing business as Guardefy, is an independent smart contract security researcher with a track record of finding numerous issues in audit contests, bugs bounties and private solo audits. His public findings and results are available at the following link:

<https://audits.sherlock.xyz/watson/panprog>

2. Disclaimer

Smart contract audit is a time, resource and expertise bound effort which doesn't guarantee 100% security. While every effort is put into finding as many security issues as possible, there is no guarantee that all vulnerabilities are detected nor that the code is secure from all possible attacks. Additional security audits, bugs bounty programs and on-chain monitoring are strongly advised.

This security audit report is based on the specific commit and version of the code provided. Any modifications in the code after the specified commit may introduce new issues not present in the report.

3. Scope of the audit

The code at the following link was reviewed:

<https://github.com/TempleDAO/origami>

commit hash: 026c132e61b2bef9179845b3dade5410bb4eb7a7

Files in scope:

```
./contracts/common/bundler/OrigamiBundler.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginCore.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginMultiAccess.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginEntryPoint.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginOhmStaking.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginTbsV1.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginTbsV2.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginKyberSwap.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginAaveV3Flash.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginMorphoFlash.sol
./contracts/common/bundler/plugins/OrigamiBundlerPluginPendleSwap.sol
```

Additionally, the following library was reviewed:

<https://github.com/TempleDAO/pendle-router-scaling>

commit hash: 4b96db96c326da6ba4d93b72ee526a93f334a5f4

Files in scope:

```
./src/PendleRouterScalingLib.sol
```

4. Audit Timeline

Audit start: November 5, 2025

Audit report delivered: November 10, 2025

Fixes reviewed: November 11, 2025

5. Findings

5.1. High severity findings

None found.

5.2. Medium severity findings

None found.

5.3. Low severity findings

5.3.1. [L-1] Malicious bundle can block all unapprove attempts by reverting `OrigamiBundlerPluginMultiAccess.setBundlerApproved` in the `supportsInterface` function.

Admin of the MultiAccess bundler plugin can approve or unapprove bundlers allowed to call the plugin. One of the reasons why the unapproval might be needed is if the bundler is compromised / becomes malicious (for example, admin keys which allow to upgrade the bundler contract are stolen). If the malicious bundler is modified to revert or return false in `supportsInterface` function, this will revert all attempts to unapprove this malicious bundler.

```
function setBundlerApproved(address account, bool value) public override onlyElevatedAccess {  
    // @audit-issue when reverts or returns false, malicious bundler blocks all unapproval attempts  
    if (!ERC165Checker.supportsInterface(account, type(IOrigamiBundler).interfaceId)) {  
        revert InvalidBundler(account);  
    }  
    emit BundlerApprovedSet(account, value);  
    approvedBundlers[account] = value;  
}
```

Likelihood

Extremely unlikely. In the current code bundler is non-upgradeable, so this is not actually possible. But if there are any upgradeable bundler implementations in the future, this will become possible.

Impact

Denial of service for admin trying to unapprove malicious bundler.

Possible mitigation

Check interface support only when approving bundler. Unapprove unconditionally.

Status: Reported.

Fix Review: Fixed as suggested.

5.3.2. [L-2] Setting vault untrusted in TBS plugin `setTrustedVault` can fail and revert for malicious tokens or too many tokens added.

TBS plugin admin can set vaults trusted or untrusted via `setTrustedVault` function. One of the reasons why setting the vault untrusted might be because the vault is compromised / becomes malicious. For example, if the vault admin private key is stolen.

In such case, vault admin can add malicious token to the vault and to TBS plugin (calling any join function). After that, `setTrustedVault` function will revert when trying to set the vault untrusted, when it tries to clear malicious token approval (malicious token will revert):

```
function setTrustedVault(address vault, bool isTrusted) external override onlyElevatedAccess {
    trustedVaults[vault] = isTrusted;

    // Unapprove all token spend
    if (!isTrusted) {
        EnumerableSetLib.AddressSet storage tokens = _approvedTokens[vault];
        uint256 length = tokens.length();
        // @audit-issue compromised vault might add too many tokens with approval, requiring more
        gas than block limit
        for (uint256 i; i < length; ++i) {
            // @audit-issue compromised vault might add malicious tokens which will revert on
            approval, preventing the vault to be set untrusted
            IERC20(tokens.at(i)).safeApprove(vault, 0);
        }
        delete _approvedTokens[vault];
    }

    emit TrustedVaultSet(vault, isTrusted);
}
```

Alternatively, malicious vault can add a lot of tokens (also adding to TBS) so that it might require too much gas to set the vault untrusted (higher than max block gas limit).

Likelihood

Extremely unlikely. Requires malicious behavior of trusted contract, but at best should still be guarded against.

Impact

Denial of service for admin trying to unapprove malicious vault.

Possible mitigation

Ignore revert of token approval when setting vault untrusted. Additionally, add cap to the number of tokens added to TBS, or allow to clear approvals up to a specified number of tokens.

Status: Reported.

Fix Review: Fixed. Setting the vault untrusted now:

1. Can skip tokens unapproval (allows to set untrusted without any other actions which can potentially revert)
2. Unapprove token revert is ignored (also, gas usage for unapprove is limited)
3. Additional function added to revoke approvals for a given list of tokens.

5.4. Informational severity findings

5.4.1. [I-1] When TBS plugins set vault to untrusted, the `_approvedTokens` set is not cleared.

When vault trusted status is set to false, it clears approval of all tokens with approvals. The list of all tokens with approvals is stored in the `_approvedTokens EnumerableSet`. Once the approvals are cleared, the set is then deleted:

```
mapping(address vault => EnumerableSetLib.AddressSet tokens) private _approvedTokens;
...
delete _approvedTokens[vault];
```

However, deleting `EnumerableSet` doesn't actually do anything: the set still remains unchanged due to internal implementation: the set hashes slot of the set storage variable to find the actual slot where data is stored, actual value of set storage variable is ignored. And `delete` only sets this storage variable to 0, but since it's ignored anyway, it doesn't change the set itself.

Proof of concept

Add this to `OrigamiBundlerPluginTbsV2.t.sol`:

```
import { EnumerableSetLib } from "solady/utils/EnumerableSetLib.sol";
import { console2 } from "forge-std/console2.sol";
...
function test_setClear() public {
    address vault = 0x000000000000000000000000000000000000000000000000000000000000000012;
    address token1 = 0x0000000000000000000000000000000000000000000000000000000000000001;
    address token2 = 0x0000000000000000000000000000000000000000000000000000000000000002;
    address token3 = 0x0000000000000000000000000000000000000000000000000000000000000003;
    address token4 = 0x0000000000000000000000000000000000000000000000000000000000000004;
    address token5 = 0x0000000000000000000000000000000000000000000000000000000000000005;
    _approvedTokens[vault].add(token1);
    _approvedTokens[vault].add(token2);
    _approvedTokens[vault].add(token3);
    _approvedTokens[vault].add(token4);
    _approvedTokens[vault].add(token5);

    // delete
    delete _approvedTokens[vault];
    console2.log("len = ", _approvedTokens[vault].length());
    address[] memory arr = _approvedTokens[vault].values();
    for (uint i = 0; i < arr.length; i++)
        console2.log("a[i] = ", arr[i]);
}
```

Impact

None. The list of approved tokens is only used to clear approvals when setting vault to untrusted. Even if the vault is set trusted again, all approvals are given correctly any time the vault function has to be called.

Possible mitigation

As there is no actual impact, simply remove the delete and keep the approved tokens set. Alternatively, clear the set one-by-one, or make it as set of set with current usage index and increase the index when the vault is set untrusted.

Status: Reported

Fix Review: Fixed as suggested by introducing nonce, which is incremented on setting vault untrusted. The list of approved tokens is taken at the nonce index, thus the set used after setting vault untrusted becomes empty.

5.4.2. [I-2] Kyber Swap plugin `swapBalance` function might not always revert when scaling input amount fails.

When KyberSwap Plugin `swapBalance` function is called, the actual amount to swap might differ slightly from the input data, it then scales input data according to token balance to swap full balance. Scaling is not always possible. If not possible, it returns `isSuccess = false` and empty data. However, for one specific case it is not clear if `isSuccess` is set to `false`, the [Kyber docs](#) say:

Note: If you set `onlyScalableSources` to true and try to scale up with a DEX that only supports scaling down, it will return empty data.

Currently only `isSuccess` is checked to revert if scaling fails:

```
bool isSuccess;  
(isSuccess, callData) = IKyberScalingHelper(SCALING_HELPER).getScaledInputData(  
    callData,  
    newSrcAmount  
);  
  
if (!isSuccess) revert ScalingFailed();
```

Since it's hard to verify what will be returned if there is DEX which only supports scaling down, it's suggested to also revert if scaled data is empty.

Possible mitigation

Revert if either `isSuccess` is `false` or scaled data is empty.

Status: Reported

Fix Review: Fixed as suggested

5.4.3. [I-3] Suggestion: `erc20TransferBalanceFromInitiator` and `permit2TransferBalanceFromInitiator` functions of Entry Point plugin will revert if there is not enough approval to transfer all balance.

When Entry Point plugin transfers full balance from the user, it will revert if not enough approval is given by the user. It makes sense to transfer all approval (up to balance) instead of balance. This might be a different function of the Entry Point plugin.

Possible mitigation

Add functions to transfer full approval up to full balance from the user to avoid unexpected reverts if approval is less than balance.

Status: Reported

Fix Review: Acknowledged

5.4.4. [I-4] Typo in comment in `PendleRouterScalingLib`

There is typo with incorrect number in the comment:

```
// 4 byte selector + 11 x 32 byte slots = 365  
sellAmountOffset = 356;
```

The actual number and calculations imply 356, but the comment has it as 365. This typo happens twice: in lines 100 and 171.

Status: Reported

Fix Review: Fixed as suggested